Operating Systems: Virtualization, Containers & Resource Isolation

Hardware Hypervisors (Type 1 vs Type 2), Linux cgroups v2, Namespaces, and OverlayFS

Doc Id: OS-VIRT-2024-007 | **Practice:** Virtualization, Cloud OS & Containers | **System:** KVM Hypervisor, Docker, Linux cgroups v2, Namespaces | **Year:** 2024

1. Virtualization Foundations: Type 1 vs. Type 2 Hypervisors

Virtualization multiplexes physical machine hardware among multiple isolated guest operating system environments.

The Virtual Machine Monitor (VMM) or Hypervisor enforces CPU, memory, and device isolation:

• **Type 1 (Bare-Metal) Hypervisors (KVM, VMware ESXi, Xen):** Execute directly on physical hardware without a host OS intermediate layer. Guest OS executes in hardware-virtualized rings. Maximum performance, low latency overhead, standard in enterprise cloud datacenters.

• **Type 2 (Hosted) Hypervisors (VirtualBox, VMware Workstation):** Run as standard application processes on top of a conventional host OS (Windows, macOS). Hardware requests pass through host OS driver stack, incurring additional abstraction layers and context switch overhead.

Architecture	Host OS Required?	Hardware CPU Access	I/O Virtualization Path	Benchmark CPU Overhead
Type 1 (Bare-Metal / KVM)	No (or embedded Linux kernel)	Intel VT-x / AMD-V hardware assist	Virtio paravirtualized / SR-IOV direct	1.2 - 2.5%
Type 2 (Hosted / VirtualBox)	Yes (e.g. Windows, macOS)	Hardware assist via host driver	Emulated software device layer	6.5 - 12.0%
OS Container (Docker / Podman)	Shared host Linux kernel	Direct native host core execution	Native host kernel block & net drivers	0.1 - 0.5%

2. Linux Container Primitives: Namespaces and Control Groups (cgroups)

Containers are NOT lightweight virtual machines; they do not virtualize hardware. A container is a standard Linux process governed by two core Linux kernel isolation primitives:

1. Linux Namespaces (Resource Visibility Isolation): Restricts what a process can see across 7 subsystem namespaces:

• **PID Namespace:** Isolates process ID tree. The container entrypoint process is PID 1 inside the container, but maps to a normal PID on the host.

• **NET Namespace:** Private virtual network interface, loopback, routing table, and iptables firewall rules.

• **MNT Namespace:** Private filesystem mount points, isolating root directory view (pivot_root).

• **IPC Namespace:** Isolated POSIX message queues, semaphores, and shared memory segments.

• **UTS Namespace:** Private hostname and domain name.

• **USER Namespace:** Maps container root UID 0 to an unprivileged non-root UID 10001 on the host for defense-in-depth.

2. Control Groups (cgroups v2 - Resource Quantity Allocation): Restricts how much of a resource a process group can consume:

• cpu.max: Enforces bandwidth quota limits (e.g. 200000 100000 specifies max 2 CPU cores per 100ms period).

• memory.max: Hard upper limit on physical DRAM. If exceeded, kernel invokes Out-Of-Memory (OOM) killer on container processes.

• io.max: Enforces read/write bytes-per-second and IOPS throttles on block devices.

3. Storage Layering: OverlayFS Mechanics

Containers achieve fast image startup and minimal disk footprint through Copy-on-Write (CoW) union filesystems, primarily **OverlayFS**.

An OverlayFS mount combines three directory hierarchies:

• **Lower Directory (lowerdir):** Read-only base container image layers. Multiple running containers share the identical read-only lowerdir layers in host storage.

• **Upper Directory (upperdir):** Read-write ephemeral container layer. All file modifications, creations, and deletions are recorded here.

• **Merged Directory (merged):** The unified unified mount point presented to the container process.

Copy-on-Write (CoW) Behavior: When a container process modifies an existing file residing in lowerdir, the kernel copies the entire file up to upperdir before applying writes. Unmodified files are read directly from shared lower layers with zero memory duplication.

Action	Source Location	Operation Performed	Upperdir Effect	Latency Impact
Read Existing File	lowerdir	Direct read from page cache	No change	Zero overhead
Create New File	upperdir	Direct write in upper directory	File created in upperdir	Native write speed
Modify Existing File	lowerdir	Copy-on-Write duplicate to upperdir	Full copy created in upperdir	One-time copy latency penalty
Delete Existing File	lowerdir	Create whiteout character device (0,0)	Whiteout node masks lowerdir	Fast metadata record